

AUTENTICAÇÃO E AUTORIZAÇÃO EM APIS COM NODE.JS

Sumário:

1. Introdução à Autenticação e Autorização em APIs.....	2
1.1 Por que a autenticação e a autorização são necessárias?.....	3
1.2 Principais riscos da ausência de autenticação e autorização em APIs.....	3
1.3 Tipos de Autenticação em APIs.....	4
1.4 Bibliotecas de Autenticação em Node.js.....	16
2. Implementação de autenticação usando JWT.....	19
2.1 Criação e Configuração Inicial de um projeto de exemplo.....	19
2.2 Criando a Base de Dados.....	21
2.3 Configuração do aplicativo Express.....	23
2.3 Configuração do servidor.....	25
2.4 Model.....	25
2.5 Service.....	26
2.6 Controller.....	28
2.7 Rotas.....	32
2.8 Middleware que valida tokens JWT.....	34
2.6 Testando as Rotas do Servidor HTTP.....	36

1. Introdução à Autenticação e Autorização em APIs

Neste material, vamos abordar dois dos pilares fundamentais da segurança no desenvolvimento de aplicações web: **autenticação** e **autorização**. Esses dois conceitos são vitais para garantir que apenas usuários legítimos acessem os dados e funcionalidades corretas em uma API (Application Programming Interface). Para começar, é essencial compreender a diferença conceitual e prática entre autenticação e autorização. Embora esses dois termos sejam frequentemente utilizados juntos, eles representam etapas distintas do processo de segurança.

Autenticação é o processo de verificar a identidade de quem está fazendo uma requisição. Em outras palavras, é o momento em que a aplicação confirma se o agente (usuário, sistema ou serviço) é realmente quem afirma ser. Essa verificação pode ocorrer de diversas formas, como por meio de credenciais (usuário e senha), tokens, certificados digitais, chaves de API ou autenticação baseada em terceiros (como Google, Facebook ou GitHub). A aplicação geralmente valida essas informações consultando uma fonte confiável, como um banco de dados ou um provedor externo de identidade. Em caso de sucesso, um mecanismo de sessão ou autenticação — como um token, cookie ou cabeçalho HTTP — é usado nas requisições subsequentes para provar que a identidade foi confirmada.

Autorização é a etapa que vem após a autenticação e tem como objetivo definir o que o usuário autenticado pode ou não fazer dentro da aplicação. Estar autenticado não garante, por si só, acesso irrestrito aos recursos do sistema. Por exemplo, um usuário pode ter realizado login com sucesso, mas ainda assim não ter permissão para acessar rotas administrativas, excluir registros ou visualizar informações confidenciais de outros usuários. A autorização, portanto, consiste na verificação de níveis de acesso, perfis (roles) ou permissões específicas associadas à identidade autenticada — geralmente embutidas em um token ou sessão. É esse controle que assegura que cada usuário só possa interagir com os recursos apropriados ao seu perfil.

Uma analogia simples, mas eficaz, que ajuda a fixar bem esses conceitos é você imaginar que está entrando em um prédio corporativo.

- Quando você mostra seu crachá na portaria e é reconhecido como funcionário, isso é autenticação: o prédio sabe quem você é.

- No entanto, o fato de ser funcionário não significa que você pode acessar todas as salas. Talvez apenas a sua sala e a copa estejam liberadas para você. Isso é autorização: o sistema de controle interno determina onde você pode entrar.

1.1 Por que a autenticação e a autorização são necessárias?

A **autenticação** permite que a API saiba quem está fazendo a requisição. Sem ela, qualquer pessoa — ou qualquer sistema externo — poderia acessar os endpoints da API sem qualquer identificação, o que abre espaço para uso indevido, fraudes, roubo de dados e ataques. Autenticar é o primeiro passo para assegurar que a comunicação esteja sendo feita com um usuário ou sistema confiável.

Após identificar quem está acessando a API, é preciso definir o que essa identidade pode fazer. A **autorização** impede que usuários comuns realizem ações sensíveis ou acessem dados confidenciais. Por exemplo, mesmo que dois usuários estejam autenticados, apenas um administrador deve poder excluir registros ou acessar painéis de controle. Sem autorização, qualquer usuário autenticado teria acesso irrestrito, o que compromete a segurança e a lógica de negócio da aplicação.

Resumidamente:

- A autenticação responde "**quem é você?**".
- A autorização responde "**o que você pode fazer?**".

Juntas, elas protegem a API contra acessos indevidos e garantem que cada operação seja realizada de forma controlada e segura.

1.2 Principais riscos da ausência de autenticação e autorização em APIs

Veja abaixo os principais riscos da ausência de autenticação e autorização em APIs:

1. **Acesso não autorizado:** Sem autenticação, qualquer pessoa pode acessar os dados ou funcionalidades da API — mesmo que sejam confidenciais, sensíveis ou críticos. Isso abre portas para invasores explorarem falhas, alterarem dados ou executarem ações administrativas.

2. **Escalada de privilégios:** Se a autenticação ou autorização for mal configurada, um usuário comum pode acessar áreas reservadas, como rotas de administrador, manipular dados de outros usuários ou até excluir registros importantes.
3. **Vazamento de dados:** Sem uma verificação robusta de identidade, informações como dados pessoais, financeiros, históricos de pedidos, prontuários médicos ou arquivos sensíveis podem ser expostos, violando a privacidade dos usuários.
4. **Execução de comandos maliciosos:** APIs abertas ou sem autenticação podem ser alvos de scripts automatizados (bots) que executam ataques como [injeção de SQL](#), [negação de serviço \(DoS\)](#), [ataque de força bruta \(brute force\)](#) e [fuzzing](#).
5. **Problemas legais:** Ao não proteger adequadamente dados de usuários, a aplicação pode infringir leis como a LGPD ([Lei Geral de Proteção de Dados](#)) no Brasil ou o GDPR na Europa, gerando multas e exigências de compensação judicial.

1.3 Tipos de Autenticação em APIs

Para proteger os dados e funcionalidades de uma API, é necessário adotar um mecanismo que permita identificar quem está fazendo a requisição. Essa é a função da autenticação. Ao longo da evolução da web, diversos modelos de autenticação foram propostos e aplicados. Os mais comuns são: **Basic Auth**, **Session-based Authentication** e **Token-based Authentication**. Cada um tem vantagens e limitações, e sua escolha depende do tipo de aplicação, da escalabilidade esperada e do nível de segurança desejado.

1.3.1. Basic Auth

A Autenticação Básica (Basic Auth) é um método simples de autenticação HTTP em que o cliente envia credenciais (**usuário** e **senha**) codificadas em **Base64** no cabeçalho **Authorization** das requisições.

Veja abaixo um exemplo de cabeçalho HTTP usando a autenticação Basic Auth, se o usuário for **admin** e a senha **1234**:

Authorization: Basic YWRtaW46MTIzNA==

O Base64 é um **método de codificação** que converte dados binários — como arquivos, imagens ou strings — em uma representação textual utilizando caracteres [ASCII](#). Essa conversão permite que os dados possam ser transmitidos com segurança por sistemas que não suportam conteúdo binário, como os cabeçalhos HTTP. Porém, é importante compreender que o Base64 é apenas uma forma de codificar, não de proteger. Ou seja, qualquer pessoa pode decodificar um conteúdo em Base64 com ferramentas simples — como <https://www.base64decode.org> — e obter o conteúdo original.

Veja na imagem abaixo como é fácil decodificar a string **admin:1234** que estava codificada usando o método Base64:

Decode from Base64 format

Simply enter your data then push the decode button.

YWRtaW46MTIzNA==

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

ASCII  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE**  Decodes your data into the area below.

admin:1234

1.3.1.1 Fluxo típico da autenticação

Veja abaixo um fluxo típico da autenticação por usuário e senha (Basic Auth):

1. **Usuário acessa o formulário de login:** O cliente (navegador, app ou front-end) exibe uma tela ou formulário onde o usuário insere suas credenciais (ex: e-mail e senha).

2. **Envio da requisição para o back-end:** O cliente codifica as credenciais digitadas em Base64 no formato **usuário:senha**. Após, envia a requisição HTTP (normalmente um POST para /login ou /auth/login) com o cabeçalho, tal como:
Authorization: Basic dXN1YXJpby5AZXhnbXBsZS5jb206c2VuaGE=
3. **Validação das credenciais:** O servidor decodifica o cabeçalho, e valida as credenciais comparando com o banco de dados.
4. **Resposta do servidor:** Se válidas, autoriza a requisição e retorna o recurso solicitado. Se inválidas, retorna erro HTTP (401 Unauthorized ou 403 Forbidden) com uma mensagem explicativa.
5. **Requisições seguintes:** O cliente repete o envio do cabeçalho Authorization em todas as requisições subsequentes.

1.3.1.2 Vantagens de Desvantagens

Observa-se que o método de autenticação básica (Basic Auth), embora seja simples de implementar e amplamente suportado, **não oferece segurança adequada por padrão**. As credenciais são apenas codificadas em Base64, o que as torna facilmente decodificáveis. Se não forem transmitidas por meio de uma conexão segura (HTTPS), podem ser interceptadas por terceiros com facilidade. Além disso, as credenciais são enviadas no cabeçalho de todas as requisições, expondo constantemente o usuário e a senha e aumentando o risco de vazamento. O Basic Auth também não oferece controle de sessão, ou seja, não há como realizar logout, revogar o acesso ou limitar a validade das credenciais. Isso o torna pouco adequado para aplicações modernas que exigem maior controle, segurança e escalabilidade. Desta forma, recomenda-se usar a Basic Auth **em ambientes internos e seguros**, onde a API é usada por poucos clientes controlados.

1.3.2. Session-based Authentication

A autenticação baseada em sessão é um dos modelos mais tradicionais de controle de acesso em aplicações web, ainda amplamente utilizada — especialmente em arquiteturas com renderização do lado do servidor (**server-side rendering**), comuns em frameworks e linguagens como PHP, Java EE, Ruby on Rails, ASP.NET, entre outras. Esse modelo é particularmente

indicado para aplicações monolíticas e sistemas legados, nos quais o backend centraliza a lógica de autenticação e mantém o estado das interações com o usuário ao longo da navegação.

1.3.2.1 Fluxo típico

Veja abaixo um fluxo típico utilizando o método Session-based:

1. **Login:** O cliente (geralmente um navegador) envia as credenciais — como e-mail e senha — ao servidor por meio de uma requisição HTTP.
2. **Validação:** O servidor valida essas credenciais consultando o banco de dados.
3. **Criação da sessão:** Se forem válidas, o servidor cria uma sessão única identificada por um **sessionId**, armazenando no lado do servidor as informações do usuário.
4. **Envio do cookie:** O **sessionId** é enviado ao cliente em forma de **cookie HTTP**.
5. **Requisições seguintes:** O navegador passa a incluir automaticamente esse cookie em cada nova requisição feita ao mesmo domínio.
6. **Verificação da sessão:** O servidor localiza a sessão associada ao **sessionId**, verifica se está ativa e, se for o caso, permite o acesso ao recurso protegido.

Durante esse processo, o servidor mantém uma estrutura de sessão associada a cada usuário autenticado. Essa sessão armazena informações temporárias relevantes — como o ID do usuário, e-mail, permissões, papel (role), e até preferências personalizadas. Esses dados ficam protegidos no lado do servidor (em memória, banco ou cache como Redis) e não são enviados diretamente ao cliente. Em vez disso, o cliente mantém apenas o identificador da sessão, o **sessionId**, como uma chave de acesso aos dados guardados no servidor. Esse identificador é geralmente enviado no cookie de resposta com atributos de segurança como [HttpOnly](#) (que impede acesso via JavaScript) e [Secure](#) (que garante envio apenas por conexões HTTPS).

Ao receber esse cookie, o navegador o armazena localmente e o inclui automaticamente em todas as requisições seguintes. Assim, o servidor consegue reconhecer o usuário, recuperar os dados da sessão e personalizar ou autorizar respostas de forma contínua. Esse modelo é classificado como **stateful**, pois exige que o servidor mantenha o estado de autenticação entre as requisições. Em outras palavras, o servidor "lembra" do usuário ao longo da navegação,

oferecendo uma experiência fluida — como manter o usuário logado por um período, conservar itens no carrinho de compras, ou navegar entre várias páginas sem exigir nova autenticação.

1.3.2.2 Vantagens de Desvantagens

Entre suas principais **vantagens**, estão a simplicidade e a boa integração com aplicações web tradicionais, já que os navegadores cuidam automaticamente do gerenciamento de cookies. Também permite controle direto sobre a sessão, incluindo expiração por inatividade, logout, e revogação imediata de acesso. Além disso, é ideal para personalização da experiência do usuário, pois o servidor pode manter informações temporárias que influenciam diretamente a interface e a navegação.

Por outro lado, a autenticação baseada em sessão apresenta **desvantagens** relevantes, especialmente em ambientes modernos e escaláveis. Por manter estado no servidor, ela exige o armazenamento de sessões ativas, o que pode comprometer o desempenho em arquiteturas distribuídas (com múltiplos servidores), exigindo soluções como replicação de sessão ou armazenamento centralizado. Além disso, por depender de cookies, exige cuidados rigorosos com segurança, como o uso obrigatório de HTTPS e a configuração adequada de atributos como HttpOnly, Secure e SameSite. Por esses motivos, embora ainda seja muito usada em sistemas tradicionais, a autenticação baseada em sessão tem sido progressivamente substituída em APIs RESTful modernas por abordagens stateless, como o uso de tokens JWT, que oferecem mais flexibilidade e escalabilidade.

1.3.3. Token-based Authentication

A autenticação baseada em **token** é uma abordagem moderna e amplamente adotada em sistemas web, APIs RESTful e aplicações móveis para verificar a identidade de usuários de forma segura, escalável e independente de estado (**stateless**). Diferentemente da autenticação baseada em sessão, que exige que o servidor armazene e mantenha o estado das interações, o modelo baseado em token transfere essa responsabilidade para o próprio cliente, por meio de um token de acesso — geralmente um **JWT** (JSON Web Token) — assinado criptograficamente com uma **chave secreta**.

1.3.3.1 Fluxo típico

Veja abaixo um fluxo típico utilizando o método Token-based:

1. **Login:** O cliente envia suas credenciais (por exemplo, e-mail e senha) à API via requisição HTTP.
2. **Validação:** O servidor autentica as credenciais consultando o banco de dados.
3. **Geração do token:** Se válidas, o servidor gera um token assinado contendo dados do usuário (como ID, e-mail, role e tempo de expiração).
4. **Envio ao cliente:** O token é devolvido ao cliente, que o armazena (geralmente em [localStorage](#), [sessionStorage](#) ou em memória).
5. **Uso do token:** Em cada requisição protegida, o cliente inclui o token no cabeçalho **Authorization: Bearer <token>**.
6. **Verificação:** O servidor valida a assinatura e o conteúdo do token. Se for legítimo e ainda estiver válido, o acesso é concedido sem necessidade de consultar novamente o banco de dados.

Essa abordagem é altamente eficiente para ambientes distribuídos, como aplicações em nuvem, microserviços ou sistemas com múltiplas instâncias de servidores, pois elimina a necessidade de gerenciamento de sessões persistentes, promovendo maior escalabilidade e desempenho.

1.3.3.2 Estrutura de um JWT

O JWT (JSON Web Token) é um padrão aberto (RFC 7519) utilizado para representar de forma segura informações entre duas partes — normalmente entre o cliente e o servidor — usando uma estrutura compacta, auto-contida e assinada digitalmente.

O token é composto por três partes separadas por pontos:

HEADER.PAYLOAD.SIGNATURE

Onde cada uma dessas partes é codificada em Base64URL (uma variação do Base64).
Veja abaixo um exemplo de um token JWT:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.XbPfbIHM
I6arZ3Y922BhjWgQzWXcXNrZ0ogtVhfEd2o

Vamos detalhar cada um destes três componentes:

- **Header:** Contém metadados como o algoritmo de criptografia (ex: HS256) usado para assinar o token e o tipo do token ("typ": "JWT"). Por exemplo:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- **Payload:** O payload contém as **claims** (declarações), que são os dados sobre o usuário ou o sistema. O payload de um token JWT pode incluir tanto **claims padronizados** quanto **claims personalizados**. Entre os claims padronizados, destacam-se: **sub** (subject), que representa o identificador do usuário; **iat** (issued at), que indica a data e hora em que o token foi emitido; e **exp** (expiration), que define o momento exato em que o token se tornará inválido. Além dessas informações básicas, o payload também pode conter claims customizados, que são definidos conforme a necessidade da aplicação — como o papel do usuário (role), endereço de e-mail, permissões específicas ou qualquer outro dado relevante para a lógica de autenticação e autorização do sistema, por exemplo:

```
{  
  "userId": 42,  
  "email": "usuario@ifrs.edu.br",  
  "role": "admin",  
  "iat": 1716066700,  
  "exp": 1735689600 // Timestamp de expiração (ex: 1 hora)  
}
```

Atenção: o payload não é criptografado, apenas codificado. Qualquer pessoa com o token pode ver seu conteúdo, por isso nunca inclua dados sensíveis, como senhas.

- **Signature:** A assinatura garante a integridade e autenticidade do token. É gerada aplicando um algoritmo de criptografia (como HMAC SHA-256) sobre o **header** e o **payload** codificados, junto com uma chave secreta (**SECRET_KEY**) conhecida apenas pelo servidor:

```
HMACSHA256(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  SECRET_KEY  
)
```

A chave secreta (secret) é um dado confidencial conhecido apenas pelo servidor e fundamental para garantir a segurança do JWT. Ela é utilizada na assinatura digital do token, assegurando que seu conteúdo não possa ser alterado sem ser detectado. Qualquer modificação, mesmo mínima, invalida a assinatura, tornando o token automaticamente rejeitado pelo servidor. Por esse motivo, a chave secreta deve ser rigorosamente protegida, mantida fora do código-fonte e armazenada em locais seguros, como variáveis de ambiente (.env).

1.3.3.3 Vantagens de Desvantagens

A autenticação baseada em tokens oferece diversas **vantagens**, especialmente em arquiteturas modernas e distribuídas. Por ser um modelo stateless, o servidor não precisa manter o estado da autenticação entre as requisições, o que favorece a escalabilidade horizontal e torna essa abordagem ideal para APIs RESTful e microsserviços, onde o token pode ser validado por qualquer instância da aplicação. Além disso, trata-se de uma solução altamente portátil, permitindo que o mesmo token seja utilizado por aplicações web, móveis ou serviços externos. Outro ponto positivo está no controle de expiração e escopo, pois os tokens podem ser configurados com validade limitada e permissões específicas — como acesso somente de leitura ou privilégios administrativos —, o que permite maior flexibilidade no controle de acesso.

No entanto, essa abordagem também apresenta **desvantagens** relevantes. A principal delas é a dificuldade de revogação: como o servidor não mantém sessões, não há um

mecanismo nativo para invalidar um token antes de seu vencimento, a não ser que se implementem soluções adicionais. Há ainda o risco de vazamento, já que, se um token for interceptado, ele poderá ser reutilizado até expirar — a menos que medidas de proteção, como o uso obrigatório de HTTPS, estejam corretamente implementadas. Outro desafio importante está no armazenamento do token no cliente, que requer cuidados específicos: por exemplo, armazená-lo em localStorage pode expô-lo a ataques do tipo [Cross-site Scripting \(XSS\)](#), tornando indispensável o uso de práticas seguras no desenvolvimento da aplicação.

1.3.4. API Key

Uma **API Key** (chave de API) é um código único gerado pelo servidor e atribuído a um cliente — que pode ser um usuário, sistema ou aplicação — com o objetivo de acessar uma API. Essa chave funciona como uma espécie de senha de acesso, sendo utilizada para identificar e autenticar o consumidor da API. Para isso, ela deve ser enviada em todas as requisições, geralmente por meio do cabeçalho HTTP ou como parâmetro da URL (query string).

O uso de API Keys é comum em cenários onde não há necessidade de autenticar o usuário final, mas sim a aplicação cliente. É uma solução bastante adotada em integrações entre servidores confiáveis, automações internas e sistemas que disponibilizam dados públicos ou de baixa sensibilidade, mas que ainda exigem controle de acesso e monitoramento de uso.

Um exemplo conhecido é a [Marvel Developer API](#), uma API pública que fornece acesso a dados sobre personagens, quadrinhos, séries, criadores e histórias do universo Marvel. Para utilizar essa API, são exigidas duas chaves de autenticação: uma Public Key, enviada em cada requisição à API, e uma Private Key, utilizada para gerar um hash de segurança junto com um timestamp (ts) no lado do cliente ou servidor. Essa combinação ajuda a proteger o acesso e a evitar uso indevido da API.

1.3.4.1 Fluxo típico

Veja abaixo um fluxo típico utilizando a **API Key**:

1. O servidor gera uma chave única e a entrega ao cliente no momento do cadastro, login ou ativação do serviço.

2. O cliente armazena essa chave com segurança e a envia em cada requisição à API — normalmente no cabeçalho HTTP ou na query string.
3. O servidor valida a chave recebida e, se for válida, processa a requisição e retorna a resposta.
4. Se a chave estiver ausente, incorreta ou expirada, a requisição é negada com status como 401 Unauthorized ou 403 Forbidden.

1.3.4.2 Vantagens de Desvantagens

A autenticação por API Key apresenta como principais **vantagens** a sua simplicidade, sendo fácil de gerar, utilizar e validar, o que a torna ideal para projetos pequenos ou integrações rápidas. Permite também controle de uso, com aplicação de limites de requisição e rastreamento do consumidor da API, além de ser amplamente compatível com bibliotecas e ferramentas HTTP. É especialmente útil para comunicação entre sistemas, como no consumo de serviços de mapas, e-mails ou pagamentos.

Por outro lado, esse modelo possui **limitações** importantes: a chave identifica apenas a aplicação, não o usuário final; pode ser facilmente exposta se incluída em código visível ao público; não possui controle de escopo, expiração ou permissões embutidos; e apresenta baixa flexibilidade para cenários mais complexos de autenticação e autorização, exigindo lógica adicional para compensar essas fragilidades.

1.3.5. OAuth 2.0

O **OAuth 2.0** é um **protocolo de autorização** amplamente adotado, que permite que um aplicativo acesse recursos protegidos em nome de um usuário, sem que ele precise compartilhar sua senha diretamente com o aplicativo. Em vez disso, o usuário concede as permissões necessárias por meio de tokens de acesso temporários, emitidos por um servidor de autorização confiável.

Esse protocolo é amplamente utilizado em funcionalidades de login com redes sociais — como "Entrar com Google" ou "Entrar com Facebook" — e também em integrações entre sistemas e APIs de terceiros, como Stripe, GitHub, Dropbox e Microsoft, oferecendo uma forma segura e padronizada de autenticação e autorização.

1.3.5.1 Fluxo típico

O funcionamento básico do OAuth 2.0 segue o seguinte fluxo:

1. O usuário tenta acessar um recurso protegido em um aplicativo cliente.
2. O aplicativo redireciona o usuário para um servidor de autorização (como Google, Facebook ou um servidor próprio).
3. O usuário faz login nesse servidor e concede permissão para que o aplicativo acesse seus dados.
4. Após a autorização, o servidor redireciona o usuário de volta ao aplicativo, incluindo um **authorization code**.
5. O aplicativo troca esse código por um token de acesso (**access token**) diretamente com o servidor de autorização.
6. Com esse token, o aplicativo passa a fazer requisições à API em nome do usuário, respeitando os escopos permitidos.

1.3.5.2 Vantagens de Desvantagens

O OAuth 2.0 apresenta diversas **vantagens** em relação a métodos mais simples de autenticação. Um dos principais benefícios é a segurança, pois o usuário não precisa compartilhar suas credenciais diretamente com a aplicação cliente. Além disso, permite a delegação de acesso, ou seja, o aplicativo acessa apenas os recursos autorizados pelo usuário — como, por exemplo, "apenas leitura de e-mail" —, sem obter controle total sobre a conta. Outro ponto forte é o [suporte ao login único \(SSO\)](#), possibilitando que o mesmo login seja utilizado em múltiplas aplicações de forma integrada. A expiração e renovação de tokens também é uma vantagem, pois melhora o controle de sessão e reduz riscos de acesso indevido a longo prazo.

Por outro lado, o OAuth 2.0 traz alguns **desafios**. Sua implementação é mais complexa do que métodos como API Key ou autenticação básica, exigindo maior conhecimento técnico. Também requer a presença de servidores de autorização confiáveis, o que pode aumentar a infraestrutura necessária. Além disso, se os fluxos não forem corretamente utilizados, a segurança da implementação pode ser comprometida.

1.3.6. OpenID Connect

O **OpenID Connect (OIDC)** é um **protocolo de autenticação construído sobre o OAuth 2.0**, projetado para verificar a identidade de um usuário de forma segura, padronizada e interoperável. Enquanto o OAuth 2.0 é voltado para autorização — ou seja, conceder a um aplicativo permissão para acessar determinados dados do usuário —, o OpenID Connect adiciona uma camada de autenticação, permitindo que o aplicativo também saiba quem é o usuário.

O OIDC é amplamente adotado em sistemas modernos de login — como "Entrar com Google", "Entrar com Microsoft", entre outros —, por oferecer uma combinação eficaz de login, segurança robusta e compatibilidade com aplicações web, móveis e APIs REST.

1.3.6.1 Fluxo típico

O funcionamento básico do OpenID Connect segue o seguinte fluxo:

1. O usuário é redirecionado para um provedor de identidade (como Google, Auth0 ou Microsoft).
2. Após realizar o login, o usuário autoriza o aplicativo a acessar seu perfil.
3. O provedor de identidade emite dois tokens:
 - a. Um **Access Token**, utilizado para acessar APIs protegidas, conforme no OAuth 2.0.
 - b. Um **ID Token**, em formato JWT, que contém informações sobre o usuário autenticado.
4. O aplicativo cliente utiliza o ID Token para autenticar o usuário localmente e, se necessário, personalizar a experiência de uso.

1.3.6.2 Vantagens e Desvantagens

O OpenID Connect oferece diversas **vantagens** para aplicações modernas. Ele permite implementar login único (SSO) entre diferentes sistemas e domínios, eliminando a necessidade de múltiplos cadastros por parte do usuário. Também aumenta a segurança, uma vez que o aplicativo cliente não precisa armazenar ou gerenciar senhas. Como é baseado em padrões seguros (como tokens JWT e HTTPS), o OIDC proporciona uma camada adicional de proteção contra acessos indevidos. Além disso, sua integração com provedores externos — como Google, Microsoft, Apple e outros — torna o processo de autenticação mais ágil e familiar ao usuário final.

Por outro lado, o protocolo também apresenta alguns **desafios**. Sua implementação requer configuração adequada com o provedor de identidade, o que pode demandar maior conhecimento técnico e atenção a detalhes como redirecionamentos, escopos e validação de tokens. Há ainda a questão da dependência de infraestrutura externa, especialmente ao utilizar provedores de terceiros como Google ou Auth0. Além disso, o OpenID Connect é, em geral, mais complexo do que métodos tradicionais de autenticação baseados em usuário e senha armazenados localmente no banco de dados.

1.4 Bibliotecas de Autenticação em Node.js

Quando se trata de implementar autenticação em aplicações Node.js, existem diferentes bibliotecas disponíveis e muito utilizadas. Sua escolha depende do tipo de aplicação, arquitetura e requisitos de segurança, outras ferramentas podem ser mais adequadas.

1.4.1. Passport.js

O [Passport.js](#) é uma biblioteca de autenticação para aplicações Node.js, extremamente flexível e modular. Ela foi projetada para ser leve, não intrusiva e fácil de integrar com aplicações Express, fornecendo uma forma padronizada de implementar autenticação local (com e-mail/senha) ou via provedores externos, como Google, Facebook, GitHub, Twitter, entre outros.

O Passport.js opera com base em estratégias de autenticação. Cada estratégia é como um “plugin” que define como autenticar o usuário. Por exemplo:

- passport-local: autenticação com e-mail e senha
- passport-jwt: autenticação com token JWT
- passport-google-oauth20: login com conta Google
- passport-github: login com conta GitHub

Você pode usar múltiplas estratégias na mesma aplicação, tornando o Passport extremamente versátil.

1.4.1.1 Fluxo típico

O funcionamento básico do Passport.js segue o seguinte fluxo:

1. O usuário envia os dados de login (ex: e-mail e senha).

2. O Passport executa a estratégia configurada (ex: consulta o banco e verifica a senha).
3. Se válido, o Passport serializa o usuário e o mantém na sessão (ou gera um token, se usar JWT).
4. Em cada requisição protegida, o Passport verifica se o usuário está autenticado.
5. Se autenticado, permite o acesso à rota protegida.

1.4.1.2 Vantagens de Desvantagens

O Passport.js é uma biblioteca flexível e modular de autenticação, ideal para aplicações Express que exigem **vários métodos de autenticação** — como login local com e-mail e senha combinado com login social por Google, Facebook ou GitHub —, permitindo configurar diferentes estratégias de forma independente. Ele oferece controle total sobre o fluxo de login e gerenciamento de sessões, sendo leve e amplamente adotado na comunidade Node.js.

No entanto, apresenta alguns **desafios**, como uma documentação oficial fragmentada, configuração mais complexa ao usar JWT e a necessidade de compreender bem o funcionamento do Express, o que pode dificultar sua adoção por iniciantes.

1.4.1. Auth.js

A [Auth.js](#) é uma biblioteca moderna, flexível e extensível para autenticação em aplicações JavaScript, tanto no frontend quanto no backend. Originalmente criada como NextAuth.js, com foco exclusivo no framework Next.js, a partir da versão 5 a biblioteca evoluiu e passou a ser compatível com qualquer framework, incluindo Express.js, tornando-se uma solução mais abrangente e universal para aplicações Node.js.

A Auth.js oferece uma solução completa de autenticação, com suporte nativo a diferentes formas de login, como: login social com provedores populares (Google, GitHub, Facebook, Discord, entre outros), login via e-mail com link mágico (magic link), autenticação com credenciais tradicionais (usuário e senha), sessões persistentes, tokens JWT, além de integração com bancos de dados como PostgreSQL, MySQL, MongoDB e SQLite. Sua estrutura baseada em adapters e callbacks permite uma personalização avançada, possibilitando configurar toda a lógica de criação de contas, persistência de sessões, regras de segurança e eventos de login de forma flexível.

1.4.2.1 Fluxo típico

O funcionamento básico da Auth.js segue o seguinte fluxo:

1. O usuário acessa a rota de login configurada na aplicação;
2. A Auth.js exibe uma interface de autenticação (ou redireciona o usuário para um provedor externo);
3. Após a autenticação, é emitido um token de sessão (via cookie) ou um JWT;
4. Em requisições futuras, o servidor recupera e valida automaticamente a sessão do usuário com base nesse token.

1.4.2.2 Vantagens de Desvantagens

O Auth.js é **ideal** para desenvolvedores que desejam evitar a complexidade de construir um sistema de autenticação do zero, oferecendo suporte nativo a autenticação por JWT, sessões persistentes e múltiplos provedores de login, como Google e GitHub. É também uma excelente escolha para projetos que exigem login social, magic links e integração com ORMs e bancos de dados relacionais ou NoSQL. Além disso, proporciona uma experiência de autenticação robusta sem depender de plataformas externas, como Firebase, Auth0 ou Cognito.

Por outro lado, é importante destacar que a Auth.js ainda está em **fase de amadurecimento fora do ecossistema Next.js**, e alguns recursos podem apresentar variações de comportamento entre frameworks, como Express. Sua configuração também pode exigir conhecimento técnico sobre OAuth 2.0, callbacks e customização de fluxos, o que pode representar uma barreira para desenvolvedores iniciantes. Em cenários mais específicos, é possível que seja necessário dedicar tempo extra à leitura da documentação, especialmente para ajustar a lógica de autenticação às regras de negócio do sistema.

2. Implementação de autenticação usando JWT

Através da aplicação dos conceitos vistos nesta aula, vamos criar uma API simples com: Rota de registro (POST /register); Rota de login (POST /login); Rota protegida (GET /dashboard); Geração e validação de JWT e Uso de middleware para autenticação.

2.1 Criação e Configuração Inicial de um projeto de exemplo

Primeiro, crie e acesse uma pasta para este projeto de exemplo.

```
mkdir projeto-jwt  
cd projeto-jwt
```

Em seguida, inicie o projeto criando o arquivo **package.json**, que será responsável por gerenciar as dependências e configurações do projeto.

```
npm init -y
```

Com o projeto inicializado, instale os **pacotes essenciais** que serão usados em produção:

```
npm install express mysql2 jsonwebtoken bcryptjs dotenv cors
```

Vamos ver o papel de cada um:

- **[express](#)**: Framework web minimalista para Node.js. Permite criar rotas, servidores HTTP, middlewares e muito mais.
- **[mysql2](#)**: Driver para conectar o Node.js a um banco de dados MySQL.
- **[jsonwebtoken](#)**: Biblioteca usada para criar, assinar, verificar e decodificar JWTs.
- **[bcryptjs](#)**: Biblioteca que implementa o algoritmo **bcrypt** para hashing seguro de senhas, transformando informações em uma sequência fixa de caracteres chamada hash.
- **[dotenv](#)**: Carrega variáveis de ambiente a partir de um arquivo **.env** (como credenciais do banco e número da porta).
- **[cors](#)**: CORS (Cross-Origin Resource Sharing) é uma especificação que define como navegadores controlam o acesso a recursos entre diferentes origens.

Em seguida, instale os **pacotes apenas para desenvolvimento** neste projeto:

```
npm install --save-dev nodemon
```

O [nodemon](#) é uma ferramenta que reinicia automaticamente o servidor toda vez que você altera um arquivo no projeto.

Abra o arquivo **package.json** e, na seção "scripts", adicione o script "dev".

```
...  
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
},  
...
```

Em seguida, crie a seguinte organização de pastas e arquivos neste projeto:

```
projeto-jwt/  
├── src/  
│   ├── config/  
│   │   └── database.js  
│   ├── controllers/  
│   │   ├── auth.controller.js  
│   │   ├── protected.controller.js  
│   │   └── public.controller.js  
│   ├── middlewares/  
│   │   └── auth.middleware.js  
│   ├── models/  
│   │   └── userModel.js  
│   ├── routes/  
│   │   ├── auth.routes.js  
│   │   ├── protected.routes.js  
│   │   └── public.routes.js  
│   └── services/  
│       └── userService.js
```

Diretório principal com o código-fonte da aplicação
Configurações gerais da aplicação
Conexão com o banco de dados MySQL
Camada responsável por lidar com as requisições HTTP
Lógica de controle dos endpoints de autenticação
Lógica de controle dos endpoints protegidos
Lógica de controle dos endpoints públicos
Funções que interceptam requisições e tratam eventos específicos
Middleware que valida tokens JWT em rotas protegidas
Camada de acesso a dados (operações SQL)
Interação com a tabela users no banco de dados
Definição das rotas e associação com controllers
Lida com o processo de autenticação do usuário
Rotas protegidas: exigem token JWT válido para acesso
Rotas públicas: acessíveis sem autenticação
Camada de regras de negócio (validação, verificações, etc.)
Regras para registrar e logar usuários

```
| tests/                # Testes manuais ou automatizados da API
| | users_api_test.rest # Testes executáveis com a extensão REST Client do VS Code
|
| | app.js              # Configuração principal do Express (rotas, middlewares)
|
| .env                  # Arquivo de variáveis de ambiente (.gitignore recomendado)
| package.json          # Gerenciador de dependências e scripts do projeto
| package-lock.json     # Controle de versões exatas das dependências instaladas
| server.js             # Inicialização do servidor Express e carregamento do app
```

2.2 Criando a Base de Dados

Nesta aula, vamos criar um database chamado **projeto_jwt_db**. Para isso, execute os seguintes comandos SQL no terminal MySQL, em uma ferramenta de gerenciamento de banco de dados (como MySQL Workbench), ou em uma extensão do VS Code (como SQLTools):

```
-- Cria o banco de dados se não existir
CREATE DATABASE IF NOT EXISTS projeto_jwt_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

-- Usa o banco de dados
USE projeto_jwt_db;

-- Cria a tabela de usuários
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insere dois tipos de usuários
INSERT INTO users (email, password, role) VALUES
('usuario@ifrs.edu.br',
```

```
'$2b$10$382cEJJYi5YxSBNvWmufHeoPHX3dqIB9NP2R2XWzt/w.DnC0gmCr2', 'user'),  
( 'admin@ifrs.edu.br',  
'$2b$10$/JLXJ62EB1k1bNq0xmpvMuTLDJb6AWmZUs741gEJb4Z.J9.3kFJM.', 'admin');
```

Esse script SQL realiza a configuração inicial do banco de dados para a aplicação. Primeiramente, ele cria o banco chamado **projeto_jwt_db** caso ainda não exista, utilizando o conjunto de caracteres **utf8mb4**, que permite armazenar corretamente acentuação e emojis, e a ordenação **utf8mb4_unicode_ci**, compatível com múltiplos idiomas. Em seguida, define que o banco de dados a ser usado será esse. Após isso, cria a tabela **users** com cinco colunas: **id** (chave primária com incremento automático), **email** (com restrição de unicidade para evitar duplicações), **password**, **role** (define os papéis user ou admin) e **created_at** (data de criação automática do registro). Por fim, o script insere dois registros fictícios na tabela, úteis para testes iniciais da API.

Em seguida, realize as seguintes alterações no arquivo **.env**:

```
DB_HOST=localhost  
DB_USER=seu_usuario  
DB_PASSWORD=sua_senha  
DB_DATABASE=projeto_jwt_db  
PORT=3000  
JWT_SECRET=segredoSuperSecreto123
```

O arquivo **.env** é utilizado para armazenar variáveis de ambiente do projeto — informações externas ao código, que podem mudar dependendo do ambiente (desenvolvimento, teste, produção). Essas variáveis são carregadas automaticamente no projeto usando a biblioteca **dotenv**.

Em seguida, realize as seguintes alterações no arquivo **src/config/database.js**:

```
const mysql = require("mysql2/promise");
require("dotenv").config();

// Cria a conexão com o banco
const pool = mysql.createPool({
  host: process.env.DB_HOST,           // Endereço do servidor de banco
  user: process.env.DB_USER,           // Usuário para conectar no banco
  password: process.env.DB_PASSWORD,   // Senha do usuário
  database: process.env.DB_DATABASE,   // Nome do banco que será usado
  waitForConnections: true,            // Faz a conexão aguardar caso todas as
  conexões estejam ocupadas
  connectionLimit: 10,                 // Limita o número máximo de conexões simultâneas
  queueLimit: 0 // Número máximo de requisições enfileiradas (0 = sem limite)
});

module.exports = pool;
```

Observe neste código que criamos um pool de conexões com o banco de dados. Um pool mantém várias conexões abertas e reaproveita elas entre as requisições, o que melhora o desempenho. Limitar o número de conexões (**connectionLimit: 10**) protege o servidor de sobrecarga.

2.3 Configuração do aplicativo Express

O **app.js** é onde configuramos o Express, as rotas e os middlewares — ou seja, a lógica da aplicação. Logo, altere o arquivo **src/app.js**:

```
// Importa o módulo Express, para a criação de servidores web com Node.js
const express = require('express');

// Importa a biblioteca cors no seu projeto
const cors = require('cors');

// Importa as rotas responsáveis pela autenticação (login, cadastro, etc.)
const authRoutes = require('./routes/auth.routes');
```

```
// Importa as rotas públicas, que não requerem autenticação
const publicRoutes = require('./routes/public.routes');

// Importa as rotas protegidas, que só podem ser acessadas com um token JWT
const protectedRoutes = require('./routes/protected.routes');

// Cria uma instância da aplicação Express
const app = express();

// Habilita CORS para qualquer origem
app.use(cors());

// Habilita o servidor para receber e interpretar requisições com corpo em JSON
app.use(express.json());

// Define o prefixo '/auth' para as rotas de autenticação
app.use('/auth', authRoutes);

// Define o prefixo '/public' para rotas acessíveis sem autenticação
app.use('/public', publicRoutes);

// Define o prefixo '/protected' para rotas que exigem autenticação com JWT
app.use('/protected', protectedRoutes);

// Exporta o app para que ele possa ser utilizado por outros arquivos
module.exports = app;
```

Este código cria a estrutura principal de uma aplicação web utilizando o framework Express no Node.js. Ele começa importando o Express e três arquivos de rotas: uma para **autenticação** (auth.routes), uma para **rotas públicas** (public.routes) e outra para **rotas protegidas** por autenticação (protected.routes). Em seguida, a aplicação é inicializada com **express()** e configurada para aceitar requisições com corpo no formato JSON. Depois, o código define os caminhos base para cada grupo de rotas: **/auth** para operações de login e cadastro, **/public** para

acessos abertos sem autenticação e **/protected** para áreas que exigem token JWT. Por fim, o objeto app é exportado, permitindo que o servidor seja iniciado em outro arquivo.

OBS: O **CORS - Cross-Origin Resource Sharing (Compartilhamento de Recursos entre Origens)**, é uma especificação que define como um navegador deve permitir (ou bloquear) que uma aplicação rodando em uma origem (ex.: <http://localhost:5173>) acesse recursos hospedados em outra origem (ex.: <http://localhost:3000>). Por padrão, os navegadores bloqueiam requisições cross-origin (de origens diferentes) por segurança. O pacote **cors** facilita habilitar essas permissões no Express.

2.3 Configuração do servidor

O **server.js** é quem realmente inicializa o servidor e escuta a porta para começar a aceitar requisições. Logo, realize as seguintes alterações no arquivo **server.js**:

```
require('dotenv').config(); // Carrega variáveis de ambiente

const app = require('./src/app'); // Importa o app já configurado (rotas,
middlewares e tratamento de erros)
const PORT = process.env.PORT || 3000;

// Inicia o servidor Express para escutar na porta definida
app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

Este trecho de código é responsável por **inicializar o servidor** da aplicação. Primeiro, ele carrega as variáveis de ambiente do arquivo **.env**, permitindo configurar parâmetros como a porta de execução sem alterar o código. Em seguida, importa o aplicativo Express já configurado no **app.js**, define a porta que será usada (pegando do **.env** ou assumindo **3000** por padrão) e inicia o servidor com **listen()**, exibindo no console uma mensagem indicando que a aplicação está rodando corretamente.

2.4 Model

A camada Model é responsável por interagir diretamente com o banco de dados, realizando operações de leitura, gravação, atualização e exclusão, sem aplicar regras de negócio.

Neste contexto, realize as seguintes alterações no arquivo **src/models/userModel.js**:

```
const db = require('../config/database');
// Importa a conexão pool com o banco de dados

class UserModel {
  // Busca um usuário pelo email
  static async findByEmail(email) {
    const [rows] = await db.query('SELECT * FROM users WHERE email = ?', [email]);
    return rows[0];
  }

  // Cria um novo usuário
  static async create(user) {
    const { email, password, role } = user;
    const [result] = await db.query(
      'INSERT INTO users (email, password, role) VALUES (?, ?, ?)',
      [email, password, role]
    );
    return result.insertId; // Retorna o ID do usuário criado
  }
}
module.exports = UserModel;
// Exporta a classe UserModel para ser usada nos services
```

Este trecho define o **UserModel**, uma classe que centraliza as operações de acesso ao banco de dados relacionadas à tabela **users**. Utilizando a conexão configurada em **database.js**, ele implementa métodos assíncronos para buscar por e-mail e criar novos registros. Cada método executa uma consulta SQL correspondente e retorna os dados ou resultados da operação, promovendo a separação clara entre a camada de dados (**Model**) e as outras camadas da aplicação.

2.5 Service

A camada Service é responsável por aplicar as regras de negócio da aplicação. Neste contexto, realize as seguintes alterações no arquivo **src/services/userService.js**:

```
const bcrypt = require('bcryptjs'); // Biblioteca para criptografia de senhas
const jwt = require('jsonwebtoken'); // Biblioteca para geração de tokens JWT
const UserModel = require("../models/userModel"); // Model responsável pelo
acesso à tabela de usuários no banco

// Classe que contém os serviços relacionados ao usuário, como registro e login
class UserService {
  // Método para registrar um novo usuário
  static async registerUser(user) {
    const { email, password, role } = user;

    // Verifica se o e-mail já está cadastrado
    const existing = await UserModel.findByEmail(email);
    if (existing) {
      throw new Error('Usuário já existe');
    }

    // Criptografa a senha antes de salvar no banco
    const hashed = await bcrypt.hash(password, 10);

    // Substitui a senha original pela criptografada
    user.password = hashed;

    // Cria o novo usuário e retorna seu ID
    const id = await UserModel.create(user);

    // Retorna os dados de sucesso (sem lançar erro)
    return { message: 'Usuário registrado com sucesso', id };
  }

  // Método para autenticar o usuário e gerar token JWT
  static async loginUser({ email, password }) {
    // Busca o usuário pelo e-mail
    const user = await UserModel.findByEmail(email);
    if (!user) {
      throw new Error('Usuário não encontrado');
    }
  }
}
```

```
// Verifica se a senha fornecida é válida
const valid = await bcrypt.compare(password, user.password);
if (!valid) {
  throw new Error('Senha inválida');
}

// Gera o token JWT
const token = jwt.sign(
  { email: user.email, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '1h' }
);

// Retorna o token e o usuário para o controller
return { token, user: { email: user.email, role: user.role } };

}
}

module.exports = UserService;
```

Este código define a classe **UserService**, responsável por gerenciar funcionalidades de cadastro e login de usuários em uma aplicação. Ela utiliza a biblioteca **bcryptjs** para criptografar senhas e **jsonwebtoken** (JWT) para gerar tokens de autenticação. O método **registerUser** verifica se o e-mail já está cadastrado, criptografa a senha e cria o usuário no banco usando o **UserModel**. Já o método **loginUser** autentica o usuário ao comparar a senha fornecida com a senha armazenada, e, se válida, gera um **token JWT** com validade de uma hora. A classe é exportada para ser usada em outras partes do sistema, como nos controladores de rotas.

2.6 Controller

A camada Controller é responsável por receber a requisição HTTP, extrair os dados necessários, chamar o Service adequado e devolver uma resposta ao cliente.

Logo, realize as seguintes alterações no arquivo **src/controllers/auth.controller.js**:

```
// Importa o serviço de usuário que contém a lógica de negócio para registro e login
const UserService = require('../services/userService');

// Classe responsável por lidar com as requisições de autenticação (registro e login)
class AuthController {

  // Método estático que trata o cadastro de um novo usuário
  static async register(req, res) {
    try {
      // Chama o serviço para registrar o usuário, passando os dados da requisição
      const result = await UserService.registerUser(req.body);

      // Retorna status 201 (Criado) com os dados retornados pelo serviço
      return res.status(201).json(result);
    } catch (error) {
      // Em caso de erro (ex: usuário já existe), retorna status 409 (Conflito) com a
      // mensagem do erro
      return res.status(409).json({ message: error.message });
    }
  }

  // Método estático que trata o login do usuário
  static async login(req, res) {
    try {
      // Chama o serviço para autenticar o usuário, passando os dados da requisição
      const result = await UserService.loginUser(req.body);

      // Retorna status 200 (OK) com o token JWT
      return res.status(200).json(result); // envia { token, user }
    } catch (error) {
      // Define o status apropriado com base na mensagem de erro
      const status =
        error.message === 'Usuário não encontrado' || error.message === 'Senha
        inválida'
          ? 401 // Não autorizado
          : 500; // Erro interno do servidor

      // Retorna o status definido com a mensagem do erro
      return res.status(status).json({ message: error.message });
    }
  }
}
```

```
// Exporta o controlador para ser utilizado nas rotas de autenticação  
module.exports = AuthController;
```

Este código define a classe **AuthController**, responsável por lidar com as requisições de registro e **login** de usuários em uma aplicação. Ela utiliza métodos estáticos que chamam os serviços correspondentes na camada **UserService**, passando os dados recebidos no corpo da requisição. O método **register** tenta criar um novo usuário e retorna status 201 em caso de sucesso ou 409 caso o usuário já exista. O método **login** tenta autenticar o usuário e, se bem-sucedido, retorna um token, os dados do usuário e o status 200; caso contrário, verifica o tipo de erro e responde com status 401 (credenciais inválidas) ou 500 (erro interno). A classe é exportada para ser usada nas rotas da aplicação.

Após, realize as seguintes alterações no arquivo **src/controllers/public.controller.js**:

```
// Controlador responsável por lidar com rotas públicas da aplicação  
class PublicController {  
  
  // Método que responde à rota pública inicial  
  static home(req, res) {  
    try {  
      // Envia uma mensagem de boas-vindas sem exigir autenticação  
      return res.status(200).send('Bem-vindo à API pública!');  
    } catch (error) {  
      // Em caso de erro inesperado, retorna status 500 com a mensagem do erro  
      return res.status(500).json({ message: 'Erro ao acessar a rota pública',  
error: error.message });  
    }  
  }  
}  
  
// Exporta o controlador para ser utilizado nas rotas públicas  
module.exports = PublicController;
```

Este código define a classe **PublicController**, responsável por controlar as rotas públicas da aplicação, ou seja, aquelas que não exigem autenticação. O método estático **home** envia uma mensagem de boas-vindas ao usuário e utiliza um bloco **try...catch** para garantir que, em caso de erro, a aplicação responda com um status 500 e uma mensagem adequada.

Agora, realize as seguintes alterações no arquivo `src/controllers/protected.controller.js`:

```
// Controlador responsável por lidar com rotas protegidas por autenticação JWT
class ProtectedController {

    // Método que responde ao painel do usuário autenticado
    static dashboard(req, res) {
        try {
            // Responde com uma mensagem usando o e-mail do usuário autenticado
            return res.status(200).json({ message: `Bem-vindo ao painel,
${req.user.email}` });
        } catch (error) {
            // Em caso de erro inesperado, retorna erro interno do servidor
            return res.status(500).json({ message: 'Erro ao acessar o painel', error:
error.message });
        }
    }

    // Método exclusivo para usuários com permissão de admin
    static adminOnly(req, res) {
        try {
            // Responde com uma mensagem personalizada usando o e-mail do
administrador autenticado
            return res.status(200).json({ message: `Bem-vindo à área admin,
${req.user.email}` });
        } catch (error) {
            // Em caso de erro inesperado, retorna erro interno do servidor
            return res.status(500).json({ message: 'Erro ao acessar a área admin',
error: error.message });
        }
    }
}

// Exporta o controlador para ser utilizado nas rotas protegidas
module.exports = ProtectedController;
```

Este código define a classe **ProtectedController**, responsável por controlar as rotas protegidas que exigem autenticação com JWT. Ela possui dois métodos estáticos: **dashboard**,

que retorna uma mensagem personalizada com o e-mail do usuário autenticado, e **adminOnly**, que faz o mesmo para a área restrita a administradores. Ambos os métodos utilizam blocos **try...catch** para capturar possíveis erros e responder com status 500 em caso de falha. A classe é exportada para ser utilizada nas rotas da aplicação.

2.7 Rotas

Agora, vamos configurar as rotas de nossa aplicação. Realize as seguintes alterações no arquivo **src/routes/auth.routes.js**:

```
const express = require('express');  
// Importa o framework Express  
  
const AuthController = require('../controllers/auth.controller');  
// Importa o controller responsável por gerenciar as ações de autenticação  
// (registro e login)  
  
const router = express.Router();  
// Cria uma nova instância do roteador do Express para definir as rotas  
// relacionadas à autenticação  
  
router.post('/register', AuthController.register);  
// Define a rota POST /register que chama o método register do AuthController  
  
router.post('/login', AuthController.login);  
// Define a rota POST /login que chama o método login do AuthController  
  
module.exports = router;  
// Exporta o roteador para ser utilizado na aplicação
```

Este código define um módulo de rotas para autenticação de nossa aplicação. Ele começa importando o **AuthController**, que contém a lógica das ações de registro e login de usuários. Em seguida, cria uma instância de roteador com **express.Router()**, permitindo que as rotas sejam organizadas separadamente do arquivo principal da aplicação. Duas rotas são

definidas: uma rota **POST /register** que aciona o método register do controller para cadastrar um novo usuário, e uma rota **POST /login** que chama o método login para autenticar o usuário e retornar um token.

Após, realize as seguintes alterações no arquivo **src/routes/public.routes.js**:

```
const express = require('express');  
// Importa o framework Express  
  
const PublicController = require('../controllers/public.controller');  
// Importa o controller responsável por lidar com rotas públicas da aplicação  
  
const router = express.Router();  
// Cria uma nova instância do roteador do Express para as rotas públicas  
  
router.get('/home', PublicController.home);  
// Define a rota GET /home que chama o método home do PublicController  
  
module.exports = router;  
// Exporta o roteador configurado para ser utilizado na aplicação
```

Este código define as rotas públicas de nossa aplicação. Ele começa importando o **PublicController**, que contém a lógica da rota pública principal. Em seguida, é criada uma instância de roteador com **express.Router()**, permitindo declarar rotas de forma modular e organizada. A rota **GET /home** é registrada e está associada ao método **home** do **PublicController**, que retorna uma mensagem simples de boas-vindas.

Após, realize as seguintes alterações no arquivo **src/routes/protected.routes.js**:

```
const express = require('express');  
// Importa o framework Express  
  
const { authenticateToken, authorizeRole } = require('../middlewares/auth.middleware');  
// Importa o middleware que valida tokens JWT em rotas protegidas  
  
const ProtectedController = require('../controllers/protected.controller');  
// Importa o controller responsável por lidar com rotas protegidas por autenticação JWT  
  
const router = express.Router();
```

```
// Cria uma nova instância do roteador do Express para definir as rotas protegidas

router.get('/dashboard', authenticateToken, ProtectedController.dashboard);
// Define a rota GET /dashboard que chama o método dashboard do ProtectedController

router.get('/admin', authenticateToken, authorizeRole('admin'),
ProtectedController.adminOnly);
// Define a rota GET /admin que chama o método adminOnly do ProtectedController

module.exports = router;
// Exporta o roteador configurado para ser utilizado na aplicação
```

Este código define as rotas protegidas de nossa aplicação. Ele começa importando o **ProtectedController**, que contém os métodos responsáveis por responder a requisições que exigem autenticação com JWT. Com **express.Router()**, cria-se uma instância de roteador para organizar essas rotas separadamente do arquivo principal. A rota **GET /dashboard** é protegida pelo middleware **authenticateToken**, que valida o token JWT antes de permitir o acesso ao método **dashboard**. A rota **GET /admin** é ainda mais restrita, exigindo autenticação e também autorização de perfil por meio do middleware **authorizeRole('admin')**, que garante que apenas usuários com a função de administrador acessem o método **adminOnly**.

2.8 Middleware que valida tokens JWT

Vamos adicionar um middleware que valida tokens JWT em rotas protegidas. Realize as seguintes alterações no arquivo **src/middlewares/auth.middleware.js**:

```
const jwt = require('jsonwebtoken');
// Importa a biblioteca JWT para verificar e decodificar tokens de autenticação

// Middleware para autenticar o token JWT enviado pelo cliente
function authenticateToken(req, res, next) {
  // Recupera o cabeçalho de autorização da requisição
  const authHeader = req.headers['authorization'];

  // Extrai o token do cabeçalho no formato "Bearer <token>"
  const token = authHeader && authHeader.split(' ')[1];

  // Se não houver token, responde com status 401 (Não autorizado)
  if (!token) return res.sendStatus(401);
```

```
// Verifica e valida o token usando a chave secreta definida no .env
jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
  if (err) return res.sendStatus(403); // Token inválido: acesso proibido

  // Se o token for válido, adiciona os dados do usuário decodificados na
  requisição
  req.user = user;

  // Passa para o próximo middleware ou rota
  next();
});
}
// Middleware para autorizar o acesso com base na função (role) do usuário
function authorizeRole(role) {
  return (req, res, next) => {
    // Verifica se o usuário possui a role exigida
    if (req.user.role !== role) {
      // Se não tiver permissão, retorna status 403 (Acesso negado)
      return res.status(403).json({ message: 'Acesso negado' });
    }
    // Se autorizado, continua para a próxima função ou rota
    next();
  };
}
module.exports = { authenticateToken, authorizeRole };
// Exporta os middlewares para serem usados nas rotas protegidas
```

Este código define dois middlewares de segurança para nossa aplicação que utiliza autenticação com JWT (JSON Web Token). O primeiro middleware, **authenticateToken**, verifica se a requisição possui um token válido no cabeçalho **Authorization**. Se o token estiver ausente, responde com status 401 (não autorizado); se for inválido, responde com status 403 (proibido). Quando o token é válido, ele decodifica os dados do usuário e os anexa ao objeto **req.user**, permitindo que outras partes da aplicação tenham acesso a essas informações. Já o segundo middleware, **authorizeRole**, é uma função que recebe um papel (por exemplo, 'admin') e retorna outro middleware que verifica se o usuário autenticado possui essa permissão. Caso contrário, retorna status 403 com uma mensagem de acesso negado. Ambos os middlewares são exportados para serem usados nas rotas que exigem autenticação e autorização na aplicação.

2.6 Testando as Rotas do Servidor HTTP

Primeiramente, assegure que o servidor que está desenvolvendo esteja rodando. Para executar o servidor usando o Nodemon, digite o seguinte comando:

```
npm run dev
```

Neste curso, utilizamos a extensão [REST Client do VS Code](#) para testar os endpoints da aplicação. Neste contexto, realize as seguintes alterações no arquivo `tests/users_api_test.rest`:

```
@baseUrl = http://localhost:3000

### Rota pública (acesso livre)
GET {{baseUrl}}/public/home
Content-Type: application/json

### Cadastro de novo usuário
POST {{baseUrl}}/auth/register
Content-Type: application/json

{
  "email": "novo@ifrs.edu.br",
  "password": "senha123",
  "role": "user"
}

### Login do novo usuário (pegar o token gerado abaixo)
POST {{baseUrl}}/auth/login
Content-Type: application/json

{
  "email": "novo@ifrs.edu.br",
  "password": "senha123"
}

### Login do usuário comum (usuário já existente no banco)
POST {{baseUrl}}/auth/login
Content-Type: application/json
```

```
{
  "email": "usuario@ifrs.edu.br",
  "password": "123456"
}

### Login do administrador (usuário já existente no banco)
POST {{baseURL}}/auth/login
Content-Type: application/json

{
  "email": "admin@ifrs.edu.br",
  "password": "admin123"
}

### Acesso ao painel (rota protegida, requer token de qualquer usuário autenticado)
GET {{baseURL}}/protected/dashboard
Authorization: Bearer {{COLE_SEU_TOKEN_AQUI}}

### Acesso à área admin (rota protegida, requer token de usuário com role "admin")
GET {{baseURL}}/protected/admin
Authorization: Bearer {{COLE_TOKEN_ADMIN_AQUI}}
```

Execute estes testes e verifique que nossa aplicação está funcionando.

Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- API DOG. Autenticação Node.js Express: Conceitos, Métodos e Exemplos. Apidog, 2024. Disponível em: <https://apidog.com/pt/blog/node-js-express-authentication/>. Acesso em: 19 maio 2025.
- ARAÚJO, J. Trabalhando com autenticação e autorização em Node.js: JWT e Passport.js. Dev.to, 2022. Disponível em: <https://dev.to/jhonyaraujooficial/trabalhando-com-autenticacao-e-autorizacao-em-nodejs-jwt-e-passportjs-19ho>. Acesso em: 19 maio 2025.
- ASCII-CODE. Tabela de caracteres ASCII. Disponível em: <https://www.ascii-code.com/pt>. Acesso em: 18 maio 2025.
- AUTH0. What is single sign-on (SSO)?. Disponível em: <https://auth0.com/pt/intro-to-iam/what-is-single-sign-on-sso>. Acesso em: 18 maio 2025.
- BASE64DECODE. Ferramenta de decodificação Base64. Disponível em: <https://www.base64decode.org>. Acesso em: 18 maio 2025.
- MARVEL DEVELOPER. Authentication for developer API. Disponível em: <https://developer.marvel.com/>. Acesso em: 18 maio 2025.
- W3SCHOOLS. Referência sobre armazenamento local e de sessão no navegador. Disponível em: https://www.w3schools.com/jsref/prop_win_localstorage.asp. Acesso em: 18 maio 2025.
- W3SCHOOLS. Referência sobre armazenamento de sessão no navegador. Disponível em: https://www.w3schools.com/jsref/prop_win_sessionstorage.asp. Acesso em: 18 maio 2025.